

SAVI-IoT: A Self-Managing Containerized IoT Platform

Hamzeh Khazaei, Hadi Bannazadeh and Alberto Leon-Garcia

Department of Electrical and Computer Engineering

University of Toronto, Ontario, Canada

Emails: {hamzeh.khazaei, hadi.bannazadeh, alberto.leongarcia}@utoronto.ca

Abstract—Internet of Things (IoT) as a service is the ultimate goal of employing cloud computing paradigm for initiating IoT application scenarios. Due to the nature of IoT ecosystems, an IoT application should be distributed, programmable and autonomic; also, it requires to support heterogeneity, security and privacy by following design patterns involved in creating IoT systems. A multi-layer cloud architecture comprising of a high-capacity core center that is connected, through high speed links, to geographically distributed smart edges seem appropriate for highly distributed and heterogeneous IoT applications. Building upon our previous initiatives and inspired by the Infrastructure as Code (IoC) paradigm, in this paper, we propose and evaluate a hierarchical, programmable and autonomic IoT platform based on the microservice models. Our platform supports big data, local/edge data processing, high level of programmability and runtime autonomic management. The autonomic management system ensures the service availability, quality of service and optimized resource utilization in the whole IoT application components autonomously. The primary results affirm a promising future of our platform toward realization of IoT as a service.

I. INTRODUCTION

Current Internet of Things (IoT) application and platform design approaches model IoT solutions as layered architectures [1] with a bottom layer consisting of deployed IoT devices, a middleware layer to expose the underlying hardware in a unified manner, and a top-level including IoT gateways, cloud edges and the core cloud at top; applications may be deployed at the top three layers to execute business logic and visualize processed sensor data [2]. Such a method of application deployment is feasible only by combination of a multi-layer cloud services with well-defined IoT middleware that connects IoT devices seamlessly to IoT gateways (aka aggregators). Figure 1 shows a high-level architecture of such an architecture.

In order to be able to deliver such multi-layer applications, resource virtualization deemed necessary at all layers. Hypervisor based virtualization (HVV) is quite mature and is heavily in use in cloud. However, HVV is not flexible, portable, programmable and lightweight enough for edge and aggregator layers, in particular [1]. As a result, a lightweight alternative to the hypervisors is required in the IoT context; the container-based virtualization, also known as Operating System (OS) Level virtualization seems appropriate for addressing above-mentioned shortcomings of

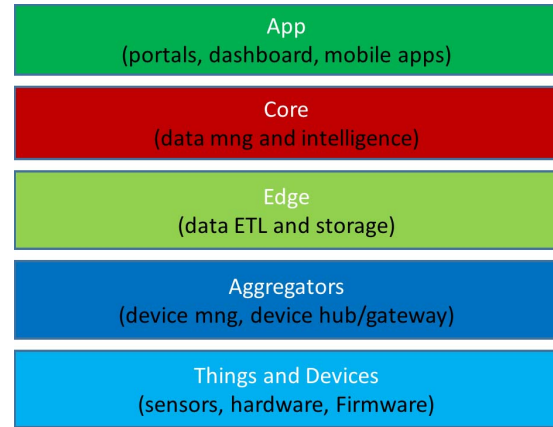


Fig. 1: A layered architecture for IoT applications.

HVV. It worth noting that, technically, containerization is not another form of virtualization with respect to HVV [3]. In essence, containers are group of isolated processes that have been configured to have limited access to resources thanks to namespace and control groups features in Linux kernel. However, containerization gives the sense of a fully isolated environment to inside applications or services as VMs do. For this reason we use ‘*isolation*’ term for containers instead. Therefore, it is more appropriate to refer HVV as *strong isolation* and containerization as *weak isolation*. Linux container isolation (LCI) can bring the same revolutions to IoT that HVV brought to cloud for resource management. LCI allows to instantiate, relocate, and optimize IoT capabilities in order to manage all resources in a more flexible and fine-grained fashion.

In this paper, we propose and evaluate a self-managing programmable IoT platform (hereafter referred to as ‘SAVI-IoT’) by leveraging both HVV and container isolation techniques to manage IoT applications end-to-end. SAVI-IoT leverages SAVI Cloud [4] as the underlying infrastructure as a service. SAVI is a two-layer academic cloud, including a core in Toronto and seven smart edges across Canada which makes it a perfect testbed for our IoT platform. While SAVI-IoT platform is influenced by layered architecture of SAVI cloud, it could be deployed on any public or private cloud. The SAVI-IoT is equipped with an autonomic

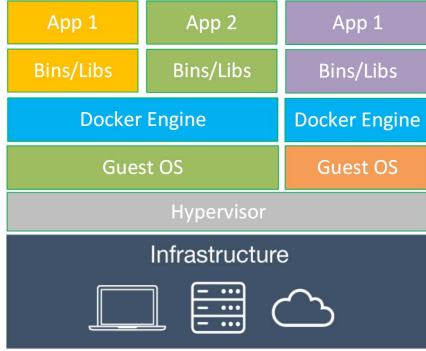


Fig. 2: Leveraging both strong and weak isolation techniques to offer microservices; adopted from [6].

management system which manages all resources at run-time autonomously to maintain performance, reliability and economics all together for IoT applications. The ultimate goal of SAVI-IoT is to provide IoT as a service for various use cases and scenarios.

This paper is organized as follows; in section II, microservice platforms will be discussed. Section III describes the SAVI-IoT platform as a special case of microservice platform for IoT scenarios. Then we elaborate on the autonomic management system that is been designed for the platform in section IV. In section V, we describe the experimental setup for evaluation of SAVI-IoT platform, the autonomic management system, in particular. We survey related work in section VI and section VII concludes the paper and highlights our future directions of research on IoT applications.

II. CLOUD MICROSERVICES

Hypervisor-based virtualization provides the highest isolated virtual environment. However, the cost of virtualization overhead is high as each VM has to run its own kernel as the Guest OS. Moreover, VM resources are mainly underutilized as each VM usually hosts one application [5]. VM virtualization limitations led to the development of Linux containers wherein only those resources will be used which are required by applications while avoiding the overhead of redundant virtualized operating systems in HVV. As a result, recently, a pattern has been adopted by many software-as-a-service providers in which both VMs and containers are leveraged to provide so called *microservices*. Microservices is an approach that allows more complex applications to be configured from basic building blocks, where each building block is deployed in a container and the constituent containers are linked together to form the cohesive application. The application's functionality can then be scaled by deploying more containers of the appropriate building blocks rather than entire new iterations of the full application.

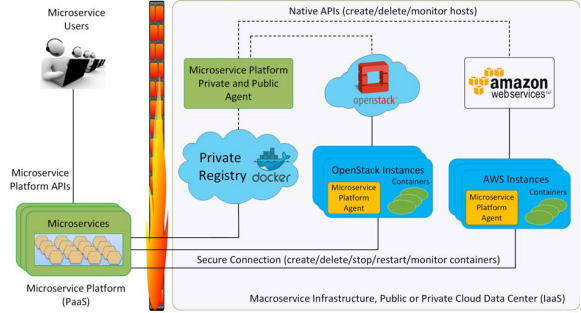


Fig. 3: High level architecture of a microservice platform; adopted from [6].

Microservice platforms (MSP) such as Nirmata¹, Docker Cloud² and Giant Swarm³ facilitate the management of such service paradigm. MSPs are automating deployment, scaling, and operations of application containers across clusters of physical machines in cloud. MSPs enable software-as-a-service providers to quickly and efficiently respond to customer demand by scaling the applications on demand, seamlessly rolling out new features and optimizing hardware usage by using only the resources that are needed (aka, continues integration and continues delivery in software engineering context). Fig. 2 shows the layered architecture in which both isolation techniques, are leveraged to deliver microservices on the cloud. Fig. 3 depicts the high-level architecture of MSPs and the way they leverage the backend public or private cloud (ie, infrastructure-as-a-service clouds). Various microservice platform providers such as Nirmata, Docker Tutum and Giant Swarm implemented their platform based on this conceptual model. However such general purpose platforms do not address the specific requirements and challenges of IoT applications and scenarios, to which we will return in sections III.

III. PLATFORM DESCRIPTION

IoT applications, regardless of their use cases, share common functional and non-functional features; they are highly distributed, interfacing numerous and heterogeneous IoT things and require multiple services for managing the big data that they are consuming. As Figure 1 shows, IoT applications require to have a bidirectional connection to IoT devices for collecting data and executing commands from/to sensors. At the aggregator layer (see Figure 1), IoT applications may need realtime processing for extract, transform or load (ETL) the data along with security and privacy measures for the underlying sensors. The Edge layer represents the closest cloud data center to the IoT plant. An IoT application may manage multiple geographically distributed but related regions. At this layer, IoT

¹<http://www.nirmata.com>

²<https://cloud.docker.com>

³<https://giantswarm.io>

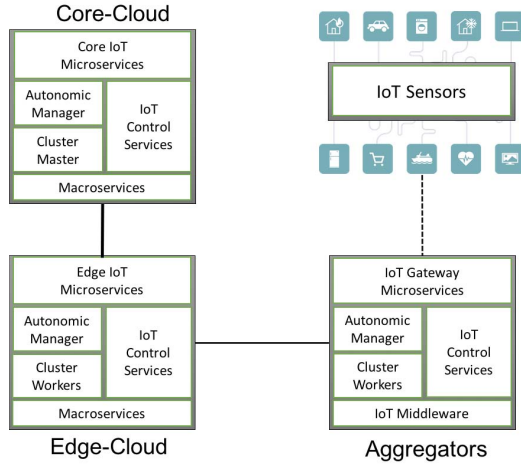


Fig. 4: SAVI-IoT: architectural view.

data for each region will be processed or stored in the closest edge cloud. The top layer, represents the cloud core data center which theoretically have unlimited resources available. That portion of the IoT applications that needs to provide a global view of the whole IoT plants for various stakeholders reside in the core. Also, in case of leveraging clustered and distributed services in the IoT application, such as big data platforms and storages, the master or manager nodes are better to be deployed at the core cloud.

Aggregators (i.e., IoT gateways) are located outside of the cloud and may be deployed on single board computers (e.g., Raspberry PIs), mini computers, or smart devices (e.g., smart phones, TVS or refrigerators) to provide management, connectivity, and data preprocessing for sensors' data. The main responsibility of IoT middleware (refer to Figure 4) is to facilitate such functionalities at aggregators. A worker node of a cluster based data processing platform may be deployed on aggregators (i.e., Cluster worker). An example for this can be a worker node of a Kafka⁴ cluster that is responsible for in-place data cleaning and aggregation. The Aggregator Autonomic Manager component is a part of the autonomic management system which is responsible for monitoring performance metrics of aggregators and scale related microservices if needs be. IoT Control Services are the control services used by the autonomic manager to scale or reconfigure microservices. IoT Gateway Microservices are the part of the IoT application that implements application functionalities at this layer. For example, in our sample implementation of the IoT application, we deploy Kafka as microservices at Aggregators.

At Edge-Cloud, we have similar orchestration of components at aggregators, though, with one difference. Here as

this part of the application is deployed on cloud, we leverage cloud macroservices (i.e., Infrastructure as a Services including VMs, networking and storage services). In our sample application, we deployed Apache Spark⁵ worker nodes for stream processing at edge cloud. Autonomic Manager and IoT Control Services have the same role as their corresponding in the aggregators.

At Core-Cloud, we have the same orchestration of components as Edge-Cloud. However, here Core IoT microservice are different. In our sample application, we deploy the master nodes of our cluster-based services due to higher reliability of Core-Cloud and the application logic. For example, we deploy the master node of our Spark cluster in the Core and Spark worker nodes at Edges. We also deploy Spark worker nodes at Core-Cloud as well. In our sample scenario, we wanted to have a global and realtime view of all sensors at top layer. We also deployed Apache Cassandra⁶ datastore as our central data storage in the Core-Cloud. The visualization service is also deployed in the Core-Cloud.

As can be seen, our IoT application comprises of multiple big data services distributed at three layers to fulfill our requirements. It goes without saying that orchestration of services at different layers is based on the IoT application requirements including both functionals and non-functionals. For delay sensitive applications, realtime processing components should be deployed closed to the sensors. Bandwidth limitation or data privacy issues also may impose local or edge processing in an IoT application. If there is no such requirements, services may be deployed on upper layers such as Edge-Cloud or Core-Cloud. As a result, in addition to functional requirements, non-functional properties such as performance, security and privacy, reliability, elasticity and scalability have a great role in service orchestration through layers.

SAVI-IoT leverages Docker-Machine, Swarm and Docker-Services for managing macroservices and microservices. Using Docker-Machine, we provision VMs on SAVI cloud (i.e. the backend cloud) at Core and Edge clouds. Then using Docker-Swarm we create a Swarm cluster of provisioned VMs. Each VM will be tagged by its role in the IoT application (e.g., manager, iot-aggregator, iot-edge-worker or iot-core-worker) and its location (e.g., Core, Edge-1, Edge-2, etc). By leveraging these tags, we are able to provide location awareness for services; more specifically, services can be deployed at only required layers and locations. For example, following command deploys a Spark worker on nodes that have *iot-edge-worker* role and are located at British Colombia (BC) edge on SAVI cloud.

```
$ docker service create --name worker
--network spark --constraint
node.labels.loc==BC --constraint
```

⁵<http://spark.apache.org>

⁶<http://cassandra.apache.org>

⁴<http://kafka.apache.org>

```
node.labels.role==iot-edge-worker
savi-iot/spark:2.1.0-hadoop-2.7
bin/spark-class
org.apache.spark.deploy.worker.Worker
spark://master-ip:7077
```

Figure 5 shows the sample IoT application that has been deployed by the SAVI-IoT platform on SAVI cloud. In this application, we have software-defined sensors that collect and report cpu, memory and network utilization of their hosting containers periodically. We also deploy the aggregators on Edge-Clouds (i.e., software-defined aggregators) as in this paper, we strive to evaluate the autonomic management system (AMS) under heavy load for which we don't have sufficient number of real and physical sensors.

IV. AUTONOMIC MANAGEMENT SYSTEM FOR IoT APPLICATIONS

The only solution to ever growing complexity of software systems is to enable them to self-manage while conforming to the end users objective. IoT software applications are one of such highly distributed and complex systems that should leverage autonomic management at runtime since manual intervention at runtime is no longer an option. As a result, we design and implement an Autonomic Management System (AMS) for applications that are being deployed by SAVI-IoT. An IoT application may comprise of multiple big data services such as a distributed NoSQL datastores (e.g., Cassandra or HBase), a sophisticate analytic platform (e.g., Apache Spark or Hadoop), light weight stream processing components (e.g., Apache Kafka or Flume), IoT middleware (e.g., Kaa⁷ project) and other distributed services depending on the use case at different layers and locations. The combination of such highly distributed systems at large scale makes the runtime environment a very complex ecosystem in which maintaining smooth and optimized operation will be a challenging task. In this paper, we propose, design and implement an AMS that optimizes the application performance while prevent under utilization of resources in the whole application stack. The AMS is based on Monitor-Analyze-Plan-Execute-Knowledge (MAPE-K) loop introduced by IBM [7]. Leveraging AMS is an attempt toward realizing DevOps or NoOps in the context of software engineering. The purpose of DevOps is to fill the gap between development and operation tasks in software development life cycle. NoOps extends the concept of DevOps in which operation tasks will be completely removed in the software lifetime. In other words, if the AMS can take care of all required operations (including but not limited to configuration, QoS, reliability, fault tolerance and protection against cyber-attacks) at runtime, then we have achieved NoOps.

The AMS proposed in this work is to deliver auto-scalability for the IoT application which brings about QoS, reliability and fault tolerance autonomously. The proposed

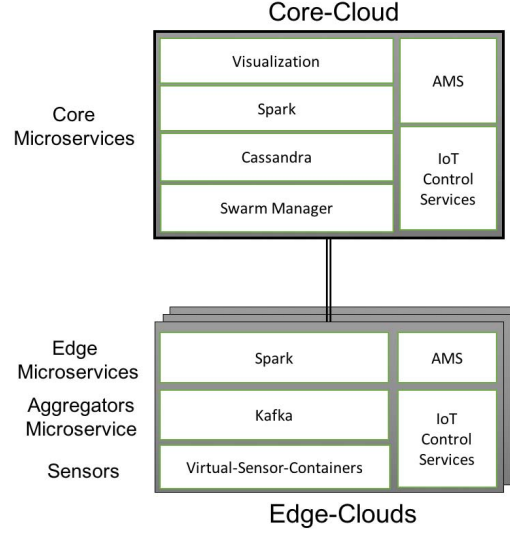


Fig. 5: Sample IoT application for the experiment.

AMS scales both microservices and macroservices in a cascading manner across all layers. As a result, the managed IoT application is scaled according to changes in workload (i.e., incoming sensor data) or changes in the internal states (e.g, node failure or network partitioning).

We change the workload by adding software-defined sensors to the application; as a result, the resource utilization (i.e., cpu, memory and network load) of aggregators increases. If resource utilization at aggregators rich a pre-defined threshold, then the aggregator microservice will be scaled out by adding more containers. Aggregator containers run on their corresponding VMs which have been set to accommodate certain number of containers. If more containers are required due to high load, the AMS will add a new VM (i.e., scaling the macroservice) at the same location and setup it up for deploying new containers. Scaling out at aggregator layer may trigger scaling at up stream services (i.e., edge services). Again, Edge services first scale out at microservice and then at macroservice level, if needs be. In a cascading manner, core micor/macro services may also be scaled out because of the expansion in underlaying services (i.e., edge services). Provided that, the whole application is scaled out at all layers to maintain QoS at high load. If load subsides, the AMS shrinks all application services (i.e., again bottom up) to maintain optimized resource utilization.

The AMS uses different criteria to scale out/in services at each layer. We adopt function f as the generic formula to scale the services.

$$f = \alpha \cdot cpu_{util} + \beta \cdot mem_{util} + \gamma \cdot net_{util} + \lambda \cdot \frac{rep_{fac}^t}{rep_{fac}^c} \quad (1)$$

⁷<https://www.kaaproject.org>

in which $\alpha + \beta + \gamma + \lambda = 1$. In addition to cpu, memory, and network utilization (percentages %), we also incorporate replication factor (i.e., rep_{fac}) for services which is the dependency factor among services. For example, defining the target replication factor (i.e., rep_{fac}^t) of Spark service (i.e. Edge service) to Kafka service (i.e., Aggregator service) as 50% means that for each Spark container there should be at least 2 Kafka containers up and running. The main idea of using the ratio of target replication factor to current replication factor (i.e., rep_{fac}^c) is to maintain QoS in times of nodes' failure or network partitioning. In other words, if due to an internal failure some of services become down or unreachable, the AMS redeploys missing services autonomically regardless of resource utilizations. The weights of parameters (i.e., $\alpha, \beta, \gamma, \lambda$) can be tuned based on a bottleneck analysis that is performed during application test. By assigning more weight to λ , we can guarantee a higher level of reliability for our application.

These four parameters give us a high degree of flexibility to define proper criteria for auto scaling. Note that at each layer, we may have different f functions as well as thresholds. Upper and lower thresholds are to trigger out-scaling and in-scaling respectively. Resource utilization for each service is the average value of all resources consumed by containers in that service.

V. EVALUATION OF THE AUTONOMIC MANAGEMENT SYSTEM

In this section, we elaborate on our experimental setup as well as results. In the experiment, we use the latest stable versions of all software components, as of March 2017, in the IoT platform and the sample application.

A. Experimental Setup

We deployed a sample IoT application using SAVI-IoT platform on SAVI Cloud. Our application leverages the Core-Cloud at University of Toronto and three of SAVI edges located at British Columbia, Ontario and Quebec provinces in Canada. The AMS, first, creates required VMs on the Core and Edge clouds. This process includes provisioning of VMs and installing Docker packages (i.e., Docker engine and Swarm). Then, a Swarm cluster will be created out of provisioned VMs; the Swarm master will be located at the Core-Cloud. All VMs will be labeled and tagged with their roles and locations. Next, the microservices will be deployed on top of macroservices at all layers; related microservices will be linked to constitute the application logic by the AMS.

As sensors, we deploy containerized virtual sensors that collect performance metrics including, CPU utilization, network load and memory consumption of their hosting containers. We refer to these containers as *virtual-sensor-container*. In other words, each virtual-sensor-container embodies three probe sensors that report resource utilizations every 15 seconds. Every virtual-sensor-container will be attached to a co-located aggregator automatically. The

Kafka service on that aggregator takes the responsibility to forward the aggregated data from virtual sensors to the upper services. Here we set the Kafka service to aggregate sensor data for every 60 seconds and then send them up to the Spark streaming service. Spark Streaming service at the Edge-Cloud aggregates all the received data streams from it's aggregators and ingest them into the Cassandra datastore located at the Core-Cloud. Edge Spark service also streams all the data from it's edge to the Spark service at the Core. Spark streaming service at the Core simply shows the whole sensor data of the entire IoT application in realtime. Figure 5 shows the experimental setup.

Note that as we discussed in section III, in most of IoT projects, aggregators and sensors are physical entities that resides outside of the cloud. However, as we mentioned before, in this paper, we leverage software-define sensors and aggregators to evaluate the efficiency of the AMS at scale.

We examine a normal shape workload to evaluate both out-scaling and in-scaling of the application. A client program requests for new sensors according to a Poisson process. Each request is translated to a virtual-sensor-container which embodies 3 virtual sensors for measuring cpu, memory and network load. The new virtual-sensor-container will be attached to an aggregator automatically. By adding more and more virtual sensors, the resource utilization at aggregators will increase. If it reaches the upper threshold, here set to 70%, according to function f_{agg} , the AMS scales out the aggregator microservice.

$$f_{agg} = 0.5 \text{ cpu}_{util} + 0.1 \text{ mem}_{util} + 0.1 \text{ net}_{util} + 0.3 \frac{rep_{fac}^t}{rep_{fac}^c} \quad (2)$$

Eq. 2 reveals that the aggregator service (i.e., Kafka) is cpu intensive as the weight of α is equal to all others combined. We use f_{edge} and f_{core} functions for the Edge and Core services.

$$f_{edge} = 0.2 \text{ cpu}_{util} + 0.5 \text{ mem}_{util} + 0.1 \text{ net}_{util} + 0.2 \frac{rep_{fac}^t}{rep_{fac}^c} \quad (3)$$

$$f_{core} = 0.2 \text{ cpu}_{util} + 0.2 \text{ mem}_{util} + 0.3 \text{ net}_{util} + 0.3 \frac{rep_{fac}^t}{rep_{fac}^c} \quad (4)$$

Spark service is memory intensive while Cassandra service is almost intensive to all resources; these sensitivities have been reflected in Eqs 3 and 4. After some time, by adding more and more sensors, the application reaches it's upper limit capacity; an upper limit capacity for a cloud application may be set for various reasons [8]. We let the application to run at full capacity for some time and then we configure the client program to remove virtual sensors

with the same Poisson process to see if the AMS shrinks the whole application accordingly.

Tables I and II show the settings for microservices and macroservices.

TABLE I: Macroservices' Settings (VM)

Layer	Services	VM (OpenStack)	Container
Aggregator	Kafka	m1.small	Type_a
Edge-Cloud	Spark	m1.medium	Type_b
	Cassandra		Type_c
Core-Cloud	Spark	m1.large	Type_b
	Cassandra		Type_c
	Visualization		Type_d

TABLE II: Microservices' Settings (Containers)

Type	Network	RAM	CPU Quota of VM
Type_a	dedicate overlay	512 MB	25.0%
Type_b	dedicate overlay	1250 MB	33.0%
Type_c	dedicate overlay	3 GB	50.0%
Type_d	dedicate overlay	1 GB	25.5%

Figure 6 shows the resource utilization of the Swarm cluster that hosts the IoT application. This figure is part of the output of the Visualization microservice.

B. Results and Discussion

We set the upper capacity limit for our application as Table III. The application has been deployed at three Edges including the Alberta, Quebec and Ontario edges and the Core cloud that is located at the University of Toronto in Canada. Due to space limitation, we only show the results for the Alberta (top) and the Quebec (bottom) edges in Figure 7. The solid lines represent the number of VMs and dashed lines show the number of containers in the application during the experiment that took around 150 minutes; we refer to each minute of the experiment as an iteration. For the first 10 minutes, application is working with initial sensors so no scaling has been initiated. It can be seen that initial configuration for each edge is one virtual-container-sensor, one Kafka container, and one Spark container (see the first 10 minute in Figure 7).

TABLE III: Upper Capacity limit for the Application

Service	VM (#)	Container per VM	Container (#)
Kafka	12	4	48
Spark	8	3	24
Cassandra	1	3	3
Visualization	1	1	1

At iteration 9, we turn the client program on to request new sensors in a round robin fashion among edges and the

core. Each request specifies the location of new sensors. For any request, the application provisions a virtual-container-sensor, that includes 3 virtual probe sensors. Then the AMS attaches the new virtual sensor to the Kafka service at the same Edge. As can be seen in the top plot of Figure 7, the AMS first scales the Kafka microservice by adding more containers as the 70% threshold has been reached according to Eq 2. Consequently, after some time, around iteration 13, the AMS scales the Spark service as well. The scaling out process of microservices is going on until around iteration 30 in which the AMS, this time, scales the macroservice, i.e., adding one VM, for the aggregator service as the existing VM is filled with containers. We can see that the macroservices scaling is also happened for the Spark service at iteration 50. This cascading scaling continues until application reaches its capacity limit around iteration 60. Afterwards, requests for new sensors will be rejected by the application.

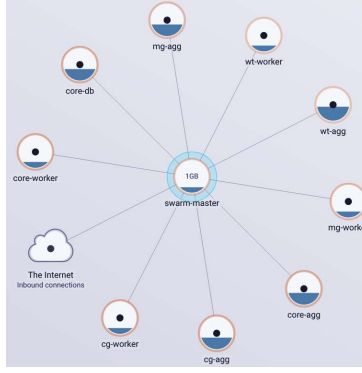
Around iteration 100, we set the client program to remove the virtual sensors with the same process. As can be seen, the AMS shrinks both microservices and macroservices to maintain optimized resource utilization. Around iteration 130, the AMS scales-in the application to the initial state as all the added sensors have been removed by the client program. The lower threshold for in-scaling has been set to 40%. The same story is going on in other Edges as can be seen from bottom plot in Figure 7.

Figure 8 shows the AMS operations on the Core services. Here, in addition to Spark and Kafka services we have data-store service (i.e., Cassandra) deployed as microservices. The top plot shows the auto-scaling at macroservices and the middle plot depicts the reconfiguration of microservices with respect to workload. As Figure 8 (top plot) shows, the AMS scales Cassandra at microservice level up to 3 instances and there is no scaling here at macroservice level. Based on our experiments, it would be beneficial to scale Cassandra datastore at microservice level for the sake of performance; however, scaling for the sake of increasing the storage capacity is better to be done at macroservice level during low load and in a scheduled manner. Adding more nodes or storage capacity to Cassandra datastore at high load, will have negative effects on application performance for a long time (even hours) due to data replication and synchronization processes in the background.

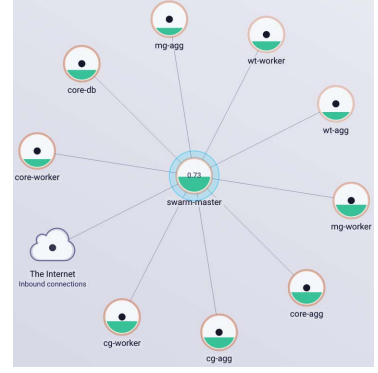
The bottom plot in Figure 8 shows the provisioning time for both macroservices and microservices. In macroservice level, provisioning means a) creating the VM at backend cloud b) installation of Docker services c) labeling node based on their roles in the application and d) joining to the application swarm cluster. As can be seen in the bottom plot (i.e., blue bars), provisioning macroservices takes 50 to 150 seconds depending on the VM specifications. In terms of microservice, the provisioning time is in order of milliseconds. Provisioning at microservice includes, loading the Docker image (images will be available locally after first instantiations) and configure it to be part of the target



(a) CPU utilization.



(b) Memory consumption.



(c) Network load.

Fig. 6: Initial resource utilization of the Swarm cluster that hosts the IoT application. This figure is the result of visualization microservice in the application.

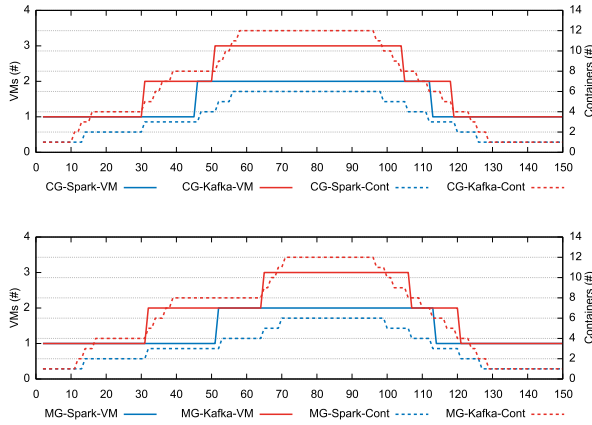


Fig. 7: Auto-scaling at both micro and macro services in the all Edges; we didn't show the number of virtual-container-sensors in the plots for the sake of clarity.

service.

It worth noting that provisioning time is different than *contribution time*. We refer to contribution time as the amount of time that is needed for the new resources (i.e., VMs or containers) to virtually contribute into the application. Contribution time is based on the application logic and the nature of the service that is being scaled. As a general rule, stateless services (e.g., load balancers) have a contribution time close to their provisioning time, while statefull services have a much longer contribution time compared to the provisioning time (i.e., distributed datastores). As a result, elasticity can be quantified based on provisioning time while scalability is more related to contribution time.

VI. RELATED WORK

IoT applications may include lightweight embedded services and enterprise services. Therefore it is necessary to

think of different ways of how to design, develop, deploy and manage such applications not only in the cloud, but also in the underlying IoT infrastructure [9]. To this end, author in [10] explored the container virtualization technology in IoT devices to analyze microservices advantages and performance in IoT applications. In particular, they discuss benefits in adopting virtualization techniques in IoT scenarios both in terms of cloud service management and business opportunities.

Pizolli et al. [11] designed and developed Cloud4IoT that is platform offering automatic deployment, orchestration and dynamic configuration of IoT support software components through microservices. This work is similar to our approach in this paper as we also leverage microservices with automatic deployment and configuration. However, their platform lacks the AMS which takes care of the whole platform at runtime autonomically. To the best of our knowledge, there is no published work in which an autonomic management system had been designed and evaluated for IoT platforms or application.

Barna et al. [8] designed an evaluated a model-based AMS for management big data applications comprising of both micro/macro services. Big data services are part of a typical IoT application. Our work in this paper incorporated all required IoT services including big data services. We leveraged some of design patterns presented in [12] in developing our AMS in this paper.

VII. CONCLUSIONS

In this paper, we proposed and evaluated SAVI-IoT, a programmable self-managing IoT platform based on microservices. The platform is generic enough to be tailored and customized for various IoT use cases. Big data compatibility, in-place data processing, high level programmability, elasticity, fault tolerance and auto-scalability are among prime features of the presented IoT platform. We deployed a sample IoT application and examined the autonomic management system under high loads of sensor data. The

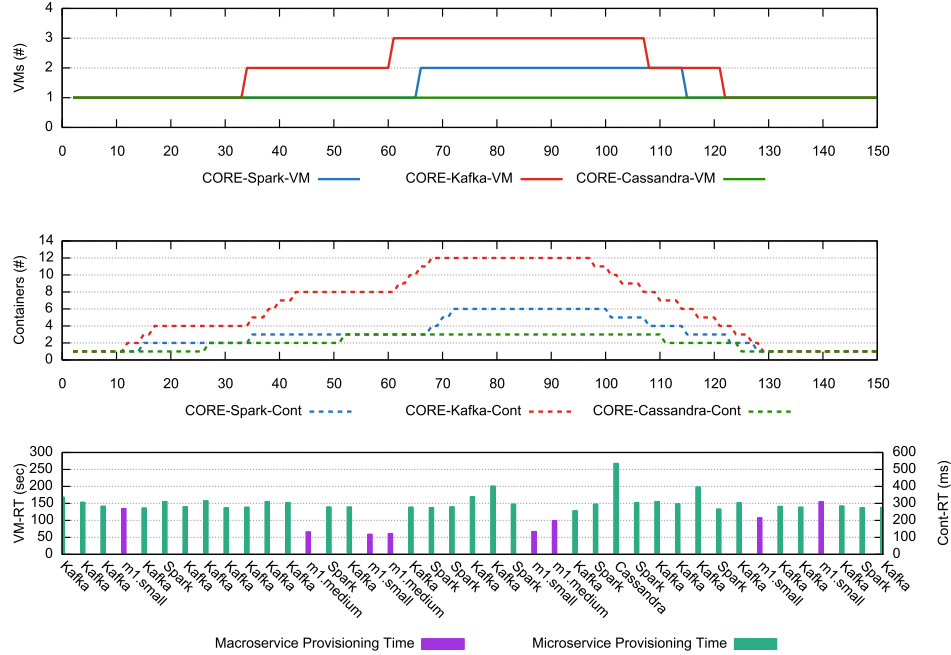


Fig. 8: Auto-scaling at both micro and macro services in the Core-Cloud; the bottom plot shows the provisioning time for VMs and containers.

primary results show a promising future for SAVI-IoT to be a cornerstone in providing IoT as a service paradigm.

As the future work, we plan to facilitate the connection and management of physical sensors to IoT applications in a software defined manner. This way sensors can be added/removed to/from the application in a plug-and-play fashion. Another direction would be implementing a mechanism to authenticate sensors and provide required security measures for IoT devices at aggregators' level. To this end, we plan to extend the autonomic management system to leverage machine learning models, queuing systems and software defined networking capabilities to provide self-protection for the SAVI-IoT platform which will be inherited by all the resulted applications.

ACKNOWLEDGMENTS

This research was supported the Natural Sciences and Engineering Council of Canada (NSERC), and the Ontario Research Fund for Research Excellence under the Connected Vehicles and Smart Transportation (CVST) project.

REFERENCES

- [1] T. Renner, M. Meldau, and A. Kliem, "Towards container-based resource management for the internet of things," in *International Conference on Software Networking (ICSN)*. IEEE, 2016, pp. 1–5.
- [2] S. Li, L. Da Xu, and S. Zhao, "The internet of things: a survey," *Information Systems Frontiers*, vol. 17, no. 2, pp. 243–259, 2015.
- [3] M. Vögler, J. Schleicher, C. Inzinger, S. Nastic, S. Sehic, and S. Dustdar, "Leonore—large-scale provisioning of resource-constrained IoT deployments," in *IEEE Symposium on Service-Oriented System Engineering (SOSE)*. IEEE, 2015, pp. 78–87.
- [4] SAVI. (2015, June) Smart Applications on Virtual Infrastructure, <http://www.savinetwork.ca>. Cloud platform.
- [5] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and linux containers," *technology*, vol. 28, p. 32, 2014.
- [6] H. Khazaei, C. Barna, N. Beigi-Mohammadi, and M. Litoiu, "Efficiency analysis of provisioning microservices," in *IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 2016, pp. 261–268.
- [7] "An architectural blueprint for autonomic computing," IBM, Tech. Rep., 2005.
- [8] C. Barna, H. Khazaei, M. Fokaefs, and M. Litoiu, "A devops architecture for continuous delivery of containerized big data applications," in *International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. IEEE, 2017.
- [9] H. Bannazadeh, A. Tizghadam, and A. Leon-Garcia, "Smart city platforms on multitier software-defined infrastructure cloud computing," in *Smart Cities Conference (ISC2), 2016 IEEE International*. IEEE, 2016, pp. 1–6.
- [10] A. Celesti, D. Mulfari, M. Fazio, M. Villari, and A. Puliafito, "Exploring container virtualization in IoT clouds," in *IEEE International Conference on Smart Computing (SMARTCOMP)*. IEEE, 2016, pp. 1–6.
- [11] D. Pizzolli, G. Cossu, D. Santoro, L. Capra, C. Dupont, D. Charalampos, F. De Pellegrini, F. Antonelli, and S. Cretti, "Cloud4IoT: a heterogeneous, distributed and autonomic cloud platform for the IoT," in *IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 2016, pp. 476–479.
- [12] S. Qanbari, S. Pezeshki, R. Raisi, S. Mahdizadeh, R. Rahimzadeh, N. Behinaein, F. Mahmoudi, S. Ayoubzadeh, P. Fazlali, K. Roshani et al., "IoT design patterns: Computational constructs to design, build and engineer edge applications," in *IEEE First International Conference on Internet-of-Things Design and Implementation*. IEEE, 2016, pp. 277–282.